

SCRIPT EN BASH

BASH, SHELL ET SCRIPT

Définition Shell :

Shell est un macroprocesseur qui permet une exécution de commande interactive ou non interactive.

Définition Script :

Les scripts permettent une exécution automatique de commandes qui seraient autrement exécutées interactivement une par une.

Définition Bash :

Bash est un interpréteur de langage de commande. Il est largement disponible sur divers systèmes d'exploitation et c'est un interpréteur de commandes par défaut sur la plupart des systèmes GNU/Linux. Le nom est un acronyme pour le « Bourne-Again SHell ».

BASH, SHELL ET SCRIPT

Au cas où vous ne le sauriez pas, Bash Scripting est une compétence indispensable pour tout travail d'administration de système Linux, même s'il n'est pas implicitement demandé par l'employeur.

```
#!/bin/bash

# This script backs up my documents
# filename: backup-documents-to-nas.sh

# Parameters
sourcefolder=/home/andreas/Documents/
targetfolder=/home/andreas/diskstation/documents/

logdirectory=/home/andreas/logs/backup
logfilepath="$logdirectory/documents-to-nas.log"

# Prerequisites
# Create log directory if it does not exist
if [ ! -d "$logdirectory" ]; then
    mkdir -p "$logdirectory"
fi

# Start Sync
rsync -itru --delete --log-file=$logfilepath $sourcefolder $targetfolder
```



SHELL - DÉFINITION

Très probablement, vous êtes actuellement assis devant votre ordinateur, vous avez une fenêtre de terminal ouverte et vous vous demandez :

« Que dois-je faire avec cette chose ? »

Eh bien, la fenêtre du terminal devant vous contient shell et shell vous permet, à l'aide de commandes, d'interagir avec votre ordinateur, donc de récupérer ou de stocker des données, de traiter des informations et diverses autres tâches simples ou même extrêmement complexes. Essayez-le maintenant! Utilisez votre clavier et tapez des commandes telles que :

- date
- cal
- pwd
- ls

SHELL - DÉFINITION

Ce que vous venez de faire, c'est qu'en utilisant des commandes et un shell, vous avez interagir avec votre ordinateur pour :

- Récupérer une date et une heure actuelles (date)
- Rechercher un calendrier (cal)
- Vérifier l'emplacement de votre répertoire de travail actuel (pwd)
- Récupérer une liste de tous les fichiers et répertoires situés dans (ls).

```
benix@dell-benix /bin/aa-easyprof
# bin
# boot
# dev
# etc
# home
# lib
# lib64
# mnt
# opt
# proc
# root
# run
# sbin
# snap
# srv
# sys
# tmp
# usr
# var
# desktopfs-pk~.txt
# file
# rootfs-pkgs.txt
# core_perl 29
# lou_maketable.d 11
# site_perl 0
# vendor_perl 11
# 2to3 -> 95 B
# 2to3-2.7 95 B
# 2to3-3.9 95 B
# 4channels 13.8 K
# 7z 36 B
# 7za 37 B
# 7zr 37 B
# 411toppm 13.8 K
# [ 58.1 K
# a2x -> 38.1 K
# a2x.py 38.1 K
# a52dec 34.2 K
# aa-audit 1.47 K
# aa-autodep 1.51 K
# aa-cleanprof 1.51 K
# aa-complain 1.4 K
# aa-decode 2.76 K
# aa-disable 1.39 K
# aa-easyprof 4.39 K
#!/usr/bin/python3
# -----
# Copyright (C) 2011-2015 Canonical Ltd.
#
# This program is free software; you can redistribute it and/or
# modify it under the terms of version 2 of the GNU General Public
# License published by the Free Software Foundation.
# -----
import apparmor.easyprof
from apparmor.easyprof import error
import os
import sys

# setup exception handling
from apparmor.fail import enable_aa_exception_handler
enable_aa_exception_handler()

if __name__ == "__main__":
    def usage():
        '''Return usage information'''
```

```
> date
lun. 05 juil. 2021 19:59:31 CEST
!w / -----
> cal -y
                                     2021
    janvier          février
lu ma me je ve sa di  lu ma me je ve sa di
                        1 2 3  1 2 3 4 5 6 7
 4 5 6 7 8 9 10      8 9 10 11 12 13 14
11 12 13 14 15 16 17 15 16 17 18 19 20 21
18 19 20 21 22 23 24 22 23 24 25 26 27 28
25 26 27 28 29 30 31

    avril          mai
lu ma me je ve sa di  lu ma me je ve sa di
                        1 2 3 4  1 2
 5 6 7 8 9 10 11      3 4 5 6 7 8 9
12 13 14 15 16 17 18 10 11 12 13 14 15 16
19 20 21 22 23 24 25 17 18 19 20 21 22 23
26 27 28 29 30      24 25 26 27 28 29 30
                        31

    juillet          août
lu ma me je ve sa di  lu ma me je ve sa di
                        1 2 3 4  1
 5 6 7 8 9 10 11      2 3 4 5 6 7 8
12 13 14 15 16 17 18  9 10 11 12 13 14 15
19 20 21 22 23 24 25 16 17 18 19 20 21 22
26 27 28 29 30 31      23 24 25 26 27 28 29
                        30 31

    octobre          novembre
lu ma me je ve sa di  lu ma me je ve sa di
                        1 2 3  1 2 3 4 5 6 7
 4 5 6 7 8 9 10      8 9 10 11 12 13 14
11 12 13 14 15 16 17 15 16 17 18 19 20 21
18 19 20 21 22 23 24 22 23 24 25 26 27 28
25 26 27 28 29 30 31 29 30

!w / -----
> ls
bin boot desktopfs-pkgs.txt dev etc file
!w / -----
```

SCRIPT - DÉFINITION

Maintenant, imaginez que l'exécution de toutes les commandes ci-dessus est votre tâche quotidienne. Chaque jour, vous devez exécuter toutes les commandes ci-dessus sans faute. Pour faciliter ces tâches, les scripts deviennent intéressants.

Créer un dossier dans votre dossier d'utilisateur : *mkdir .bin* puis un sous dossier "scripts".

Pour voir ce que l'on entend par script, utilisez shell en combinaison avec votre éditeur de texte préféré. Par exemple avec *nano* ou *vi* créer un nouveau fichier appelé first.sh contenant toutes les commandes ci-dessus, chacune sur une ligne distincte.

Maintenant, rendez votre nouveau fichier exécutable à l'aide de la commande *chmod* avec l'option *u+xr*.

Enfin, exécutez votre nouveau script en préfixant son nom avec *./first.sh*

SHELL - RÉSULTAT

Comme vous pouvez le voir, en utilisant des scripts, toute interaction shell peut être automatisée et scriptée.

De plus, il est désormais possible d'exécuter automatiquement notre nouveau script shell *first.sh* quotidiennement à tout moment en utilisant le planificateur de tâches cron et de stocker la sortie du script dans un fichier à chaque fois qu'il est exécuté.

```
> ./first.sh
lun. 05 juil. 2021 20:31:21 CEST
    juillet 2021
lu ma me je ve sa di
                1  2  3  4
5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31

/home/benix/Downloads
802.11ay-vs-802.11ad.png    Discord
Autre                      Expertise
```

BASH - DÉFINITION

Jusqu'à présent, nous avons couvert le shell et les scripts.

Et Bash ? Où s'intègre le bash ?

Comme déjà mentionné, le bash est un interpréteur par défaut sur de nombreux systèmes GNU/Linux, nous l'avons donc utilisé même sans nous en rendre compte. C'est pourquoi notre script shell précédent fonctionne même sans que nous définissions bash comme interpréteur.

Pour voir quel est votre interpréteur par défaut, exécutez la commande `echo $SHELL`

```
> echo $SHELL  
/bin/zsh
```

```
#!/bin/bash
```


BASH - DÉFINITION

Il existe divers autres interpréteurs de shell disponibles, tels que *Korn shell*, *C shell* et plus encore. Pour cette raison, il est recommandé de définir l'interpréteur shell à utiliser explicitement pour interpréter le contenu du script.

Pour trouver le chemin de l'interpréteur de votre script Bash, exécutez la commande *which bash*.

Dans votre fichier à la première ligne ajoutez avec un shebang *#!/bin/bash* et insérez-le dans votre script.

À partir de maintenant, tous nos scripts incluront la définition de l'interpréteur shell *#!/bin/bash*

```
#!/bin/bash  
  
date  
cal  
pwd  
ls  
ranger
```

BASH - NOMS DE FICHIERS ET AUTORISATIONS

Vous avez peut-être déjà remarqué que pour exécuter un script shell, le fichier doit être rendu exécutable à l'aide de la commande *sudo chmod +rx NOM_DE_FICHIER*. Par défaut, tout fichier nouvellement créé n'est pas exécutable quel que soit son suffixe d'extension de fichier.

L'extension de fichier sur les systèmes GNU/Linux n'a généralement aucune signification à part le fait que lors de l'exécution de la commande *ls* pour répertorier tous les fichiers et répertoires, il est immédiatement clair que le fichier avec l'extension *.sh* est vraisemblablement un script shell et le fichier avec *.jpg* est susceptible d'être une image compressée avec perte.

Dans les systèmes GNU/Linux, la commande *file NOM_DE_FICHIER* peut être utilisée pour identifier un type de fichier.

```
~/Downloads -----  
> file 802.11ay-vs-802.11ad.png  
802.11ay-vs-802.11ad.png: PNG image data, 1004 x 971, 8-bit/color RGBA, non-interlaced  
~/Downloads -----  
> file first.sh  
first.sh: Bourne-Again shell script, ASCII text executable
```

SCRIPT - EXECUTION

Parlons d'une autre manière d'exécuter des scripts bash. Dans une vue très simpliste, un script bash n'est rien d'autre qu'un fichier texte contenant des instructions à exécuter dans l'ordre de haut en bas. La façon dont les instructions sont interprétées dépend du shebang défini ou de la façon dont le script est exécuté.

Une autre façon d'exécuter des scripts bash consiste à appeler explicitement l'interpréteur, par exemple :

```
echo date > date.sh
```

Exécutant ainsi le script sans avoir besoin de rendre le script shell exécutable et sans déclarer shebang directement dans un script shell. En appelant explicitement le binaire exécutable bash, le contenu de notre fichier date.sh est chargé et interprété comme Bash Shell Script.

BASH - CHEMIN RELATIF VS ABSOLU

La meilleure façon pour expliquer un chemin de fichier relatif ou absolu est probablement de visualiser le système de fichiers GNU/Linux comme un bâtiment à plusieurs étages. Le répertoire racine (porte d'entrée du bâtiment) " / " fournit l'entrée à l'ensemble du système de fichiers (bâtiment), donnant ainsi accès à tous les répertoires (salles) et fichiers (personnes).

GNU/Linux propose un simple outil de boussole pour vous aider à naviguer dans le système de fichiers sous la forme de la commande *pwd*.

Cette commande, lorsqu'elle est exécutée, affichera toujours votre emplacement actuel.

BASH - CHEMIN RELATIF VS ABSOLU

```
[beni@behnam-xps ~]$ cd /
[beni@behnam-xps /]$ pwd
/
[beni@behnam-xps /]$ cd home/beni/
[beni@behnam-xps ~]$ pwd
/home/beni
[beni@behnam-xps ~]$ ls
Bureau          mnt
Documents      Images          Modèles
[beni@behnam-xps ~]$ cd ..
[beni@behnam-xps home]$ pwd
/home
[beni@behnam-xps home]$ cd beni/Téléchargements/ISO/
[beni@behnam-xps ISO]$ cd ../../
[beni@behnam-xps ~]$ pwd
/home/beni
```

BASH - CODER UN SCRIPT

Écrivons un script basique en shell bash. Le but de ce script est d'imprimer :
"Crêpe au Nutella"

À l'aide de la commande *echo*. Allez dans le dossier "scripts" que vous avez créé auparavant puis tapez la commande suivante : *nano CN.sh*

Enregistrez et quittez votre fichier et rendez votre script exécutable avec la commande *chmod u+xr* et exécutez-le en utilisant le chemin relatif *./CN.sh* .

```
[beni@behnam-xps Téléchargements]$ chmod 100 CN.sh
[beni@behnam-xps Téléchargements]$ ls -l
total 1588
---x----- 1 beni root    38 12 août  20:08 CN.sh
```

Chaque MODE a la forme « [ugoa]*([-+=]([rwx]0-7)+ ».

BASH - CODER UN SCRIPT

Toute commande qui peut être exécutée avec succès directement via le terminal shell bash peut être sous la même forme que celle utilisée dans le script shell bash. En fait, il n'y a pas de différence entre l'exécution de commandes directement via un terminal ou dans un script shell en dehors du fait que le script shell propose une exécution non interactive de plusieurs commandes en un seul processus.

La plupart des commandes acceptent ce qu'on appelle des options et des arguments. Les options de commande sont utilisées pour modifier le comportement de la commande afin de produire des résultats de sortie alternatifs et sont préfixées par '-'. Les arguments peuvent spécifier la cible d'exécution de la commande telle qu'un fichier, un répertoire, du texte, etc.

BASH - CODER UN SCRIPT

Chaque commande est livrée avec une page de manuel qui peut être utilisée pour en savoir plus sur sa fonction ainsi que sur les options et les arguments acceptés par chaque commande spécifique.

Utilisez la commande *man* pour afficher la page de manuel de toute commande souhaitée. Par exemple, pour afficher une page de manuel pour la commande *ls*, exécutez *man ls*.

Développez lentement votre script en testant chaque ligne principale en l'exécutant d'abord sur la ligne de commande de votre terminal. En cas de succès, transférez-la dans votre script shell.

BASH - CODER UN SCRIPT

```
[beni@behnam-xps ~]$ ls --help
Utilisation : ls [OPTION]... [FICHIER]...
Afficher des renseignements sur les FICHIERS (du répertoire actuel par défaut).
Trier les entrées alphabétiquement si aucune des options -cftuvSUX ou --sort
ne sont utilisées.

Les arguments obligatoires pour les options longues le sont aussi pour les
options courtes.
-a, --all                ne pas ignorer les entrées débutant par .
-A, --almost-all        ne pas inclure . ou .. dans la liste
    --author              avec -l, afficher l'auteur de chaque fichier
-b, --escape             afficher les caractères non graphiques avec des
                        protections selon le style C
```

```
SSH(1)                                BSD General Commands Manual                                SSH(1)

NAME
    ssh - OpenSSH remote login client

SYNOPSIS
    ssh [-4AaCfGgKkMNnqsTtVvXxYy] [-B bind_interface] [-b bind_address] [-c cipher_spec] [-D [bind_address:]port]
    [-E log_file] [-e escape_char] [-F configfile] [-I pkcs11] [-i identity_file] [-J destination] [-L address]
    [-l login_name] [-m mac_spec] [-O ctl_cmd] [-o option] [-p port] [-Q query_option] [-R address] [-S ctl_path]
    [-w host:port] [-w local_tun[:remote_tun]] destination [command]

DESCRIPTION
    ssh (SSH client) is a program for logging into a remote machine and for executing commands on a remote machine. It is
    intended to provide secure encrypted communications between two untrusted hosts over an insecure network. X11 connections,
    arbitrary TCP ports and UNIX-domain sockets can also be forwarded over the secure channel.

    ssh connects and logs into the specified destination, which may be specified as either [user@]hostname or a URI of the form
    ssh://[user@]hostname[:port]. The user must prove his/her identity to the remote machine using one of several methods (see
    below).

    If a command is specified, it is executed on the remote host instead of a login shell.

    The options are as follows:

    -4          Forces ssh to use IPv4 addresses only.

    -6          Forces ssh to use IPv6 addresses only.

    -A          Enables forwarding of connections from an authentication agent such as ssh-agent(1). This can also be specified on
    a per-host basis in a configuration file.
```

BASH - CODER UN SCRIPT

Repreonons notre script "CN.sh" qui est un script assez basique et simple, mais nous pouvons écrire des script plus poussée pour des tâches plus intéressantes.

Par exemple un script qui va sauvegarder un dossier personnel. Pour créer le script de sauvegarde, nous allons utiliser la commande *tar* avec diverses options *-czf*

c : crée un nouveau fichier .tar

v : la progression lors de la compression

f : nom du fichier

z : filtrer l'archive à travers gzip

BASH - CODER UN SCRIPT

Écrivez le code suivant dans un nouveau fichier appelé backup.sh, rendez le script exécutable et exécutez-le :

```
#!/bin/bash
```

```
tar -czf /tmp/NOM.tar.gz /home/USER/.bin/scripts
```



BASH - CODER UN SCRIPT - VARIABLES

Les variables permettent à un développeur de stocker des données, de les modifier et de les réutiliser tout au long du script. Créez un nouveau script *yo.sh* écrivez ce bout de code dans le fichier :

```
#!/bin/bash
```

```
salut="yo"
```

```
user=$(whoami)
```

```
now=$(date "+%A %D")
```

```
echo "$Bienvenue ${user}! Nous sommes le ${now} et il fait un super beau temps!"
```

```
echo "La version de votre Bash shell est : ${BASH_VERSION}. Voilà!"
```

BASH - CODER UN SCRIPT - VARIABLES

Regardons le script de plus près. Tout d'abord, nous avons déclaré une variable 'salut' et on lui a attribué une valeur string. La prochaine variable 'user' contient une valeur de nom d'utilisateur exécutant une session shell. Cela se fait grâce à une technique appelée substitution de commandes.

Cela signifie que la sortie de la commande *whoami* sera directement affectée à la variable 'user'. Il en va de même pour notre prochaine variable 'now' qui affiche la date d'aujourd'hui avec la commande et l'argument `date +%A`.

La deuxième partie du script utilise la commande *echo* pour imprimer un message tout en substituant les noms de variables avec les préfixés \$ par leurs valeurs pertinentes.

BASH - CODER UN SCRIPT - VARIABLES

Les variables peuvent également être utilisées directement sur la ligne de commande du terminal.

Exemple : crée une variable $a=4$ et $b=8$ avec des données entières. En utilisant la commande *echo*, nous pouvons imprimer leurs valeurs ou même effectuer une opération arithmétique :

```
→ ~ a=4
→ ~ b=8
→ ~ echo $a
4
→ ~ echo $b
8
→ ~ echo $[$a + $b]
12
→ ~
```

BASH - CODER UN SCRIPT - VARIABLES

Maintenant que nous avons l'introduction de la variable bash derrière nous, nous pouvons mettre à jour notre script de backup :

```
#!/bin/bash

user=$(whoami)
input=/home/$user/Documents
output=/tmp/${user}_Documents_$(date +%Y-%m-%d).tar.gz

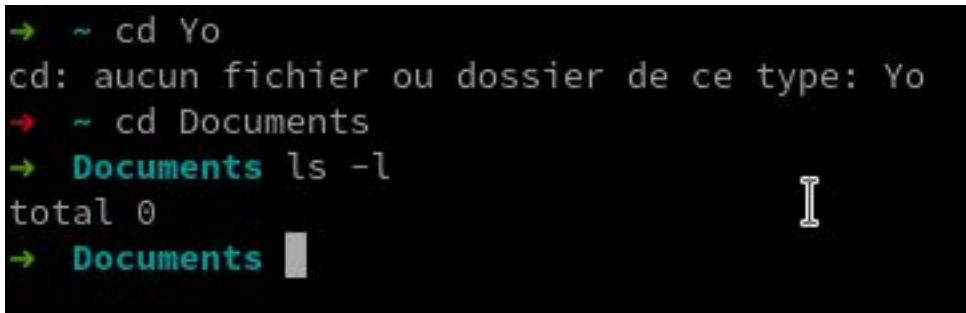
tar -czf $output $input
echo "Backup of $input terminé. Voici quelques détails:"
ls -l $output
```

BASH - INPUT, OUTPUT ET ERREURS DE REDIRECTIONS

Normalement, les commandes exécutées en ligne de commande produisent une sortie, nécessitent une entrée ou renvoient un message d'erreur. C'est un concept important pour les scripts shell ainsi que pour travailler en ligne de commande de GNU/Linux en général.

Chaque fois que vous exécutez une commande, 3 résultats possibles peuvent se produire.

1. Le premier scénario est que la commande produira une sortie attendue.
2. La commande générera une erreur
3. Votre commande pourrait ne produire aucune sortie du tout.



```
→ ~ cd Yo  
cd: aucun fichier ou dossier de ce type: Yo  
→ ~ cd Documents  
→ Documents ls -l  
total 0  
→ Documents
```


BASH - INPUT, OUTPUT ET ERREURS DE REDIRECTIONS

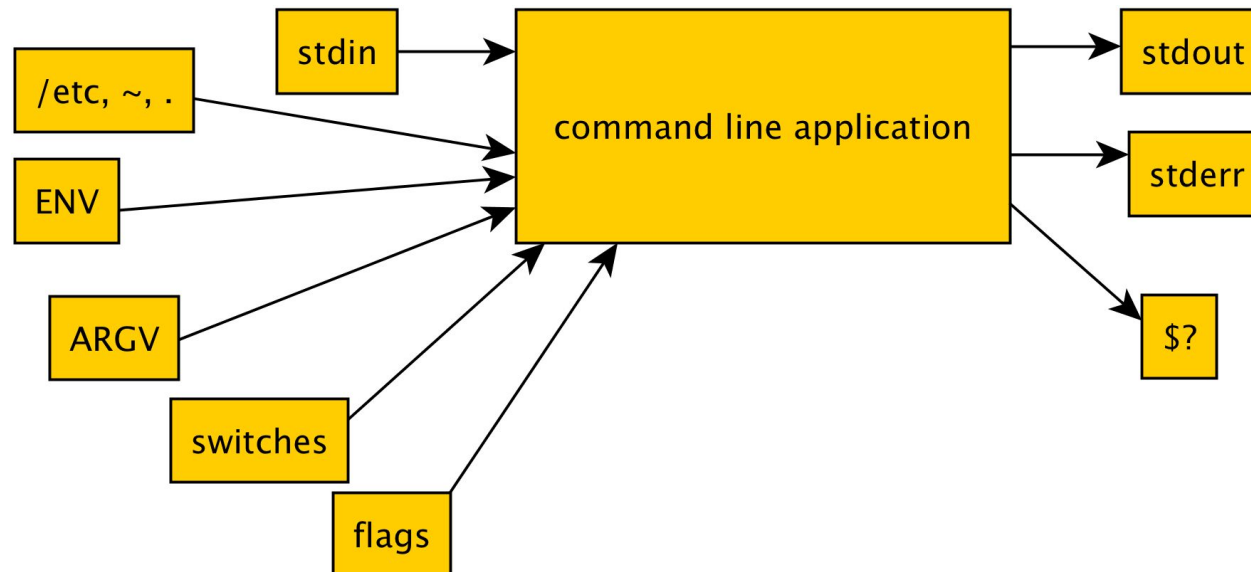
La différence entre la sortie "stdout" et la sortie "stderr" est un concept essentiel car elle nous permet de cibler une menace, c'est-à-dire de rediriger chaque sortie séparément.

La notation > est utilisée pour rediriger "stdout" vers un fichier alors que la notation 2> est utilisée pour rediriger stderr et &> est utilisée pour rediriger à la fois stdout et stderr. La commande cat est utilisée pour afficher le contenu d'un fichier donné. Voici un exemple :

```
[benix@beni-xps Bash]$ ls foobar barfoo
ls: cannot access 'foobar': No such file or directory
ls: cannot access 'barfoo': No such file or directory
[benix@beni-xps Bash]$ ls foobar barfoo > stdout.txt
ls: cannot access 'foobar': No such file or directory
ls: cannot access 'barfoo': No such file or directory
[benix@beni-xps Bash]$ ls foobar barfoo 2> stderr.txt
[benix@beni-xps Bash]$ ls foobar barfoo &> stdoutandstderr.txt
[benix@beni-xps Bash]$ cat stdout.txt
[benix@beni-xps Bash]$ cat stderr.txt
ls: cannot access 'foobar': No such file or directory
ls: cannot access 'barfoo': No such file or directory
[benix@beni-xps Bash]$ cat stdoutandstderr.txt
ls: cannot access 'foobar': No such file or directory
ls: cannot access 'barfoo': No such file or directory
[benix@beni-xps Bash]$
```

BASH - INPUT, OUTPUT ET ERREURS DE REDIRECTIONS

Lorsque vous ne savez pas si votre commande a produit stdout ou stderr, essayez de rediriger sa sortie. Par exemple, si vous êtes en mesure de rediriger sa sortie avec succès vers un fichier avec la notation `2>`, cela signifie que votre commande a produit stderr. Inversement, la redirection réussie de la sortie de la commande avec la notation `>` indique que votre commande a produit stdout.



BASH - INPUT, OUTPUT ET ERREURS DE REDIRECTIONS

Revenons à notre script backup.sh. Lors de l'exécution de notre script de sauvegarde, vous avez peut-être remarqué un message supplémentaire affiché par la commande tar :

tar : suppression du '/' de début des noms de membres

```
[benix@beni-xps Bash]$ ./backup.sh
tar: Removing leading '/' from member names
Backup of /home/benix/Documents/Scannes terminé. Voici quelques détails:
-rw-r--r-- 1 benix benix 455667 14 oct. 11:47 /tmp/benix-Scannes-2021-10-14.tar.gz
```

Malgré le caractère informatif du message, il est envoyé au stderr. En un mot, le message nous dit que le chemin absolu a été supprimé, donc l'extraction du fichier compressé n'écrasera aucun fichier existant.

BASH - INPUT, OUTPUT ET ERREURS DE REDIRECTIONS

Maintenant que nous avons une compréhension de base de la redirection de sortie, nous pouvons éliminer ce message stderr en le redirigeant avec la notation `2>` vers `/dev/null`. Imaginez `/dev/null` comme un puits de données, qui supprime toutes les données qui y sont redirigées.

```
1 #!/bin/bash
2
3 #Ce script va faire une sauvegarde dans le dossier /tmp
4
5 user=$(whoami)
6 input=/home/$user/Documents/Scannes
7 output=/tmp/${user}-Scannes-$(date +%Y-%m-%d).tar.gz
8
9 tar -czf $output $input 2> /dev/null
10 echo "Backup of $input terminé. Voici quelques détails:"
11 ls -l $output
12
13 exit $?
```

BASH - INPUT, OUTPUT ET ERREURS DE REDIRECTIONS

bash shell comporte également le nom d'entrée stdin. Généralement, la saisie du terminal provient d'un clavier. Toute frappe que vous tapez est acceptée comme stdin.

La méthode alternative consiste à accepter l'entrée de commande à partir d'un fichier en utilisant la notation `<`. Considérons l'exemple suivant où nous alimentons d'abord la commande `cat` à partir du clavier et redirigeons la sortie vers `file.txt`. Plus tard, nous autorisons la commande `cat` à lire l'entrée de `file.txt` en utilisant la notation `<` :

```
[benix@beni-xps Bash]$ cat > file.txt  
Coucou ça va ?  
[benix@beni-xps Bash]$ cat < file.txt  
Coucou ça va ?  
[benix@beni-xps Bash]$ ls  
backup.sh  file.txt
```

BASH - FUNCTIONS

Les fonctions permettent à un programmeur d'organiser et de réutiliser le code, augmentant ainsi l'efficacité, la vitesse d'exécution ainsi que la lisibilité de l'ensemble du script.

Il est possible d'éviter d'utiliser des fonctions et d'écrire n'importe quel script sans y inclure une seule fonction. Cependant, vous risquez de vous retrouver avec un code volumineux, inefficace et difficile à déboguer. Au moment où vous remarquez que votre script contient deux lignes du même code, vous pouvez envisager d'activer une fonction à la place.

```
}  
bubblesort() {  
    local size=${#dates[@]}  
    echo -e "\nArray size: $size"  
    n=$size  
    until (( n <= 0 )); do  
        newn=0  
        echo -e "\nIteration: ${size-n}"  
        for (( i=0; i < n; i++ )); do  
            if ((  
                ( ${dates[i]} > 0 ) && \  
                ( $(date -d "${dates[i]}" +%s) > $(date -d "${dates[i+1]}" +%s) )  
            )); then  
                echo swap "${dates[i+1]}" with "${dates[i]}"  
                swapDates $((i+1)) $i  
                newn=$((i+1))  
            fi  
        done  
        n=$newn  
    done  
}
```

BASH - FUNCTIONS

Vous pouvez considérer la fonction comme un moyen de regrouper le nombre de commandes différentes en une seule commande. Cela peut être extrêmement utile si la sortie ou le calcul dont vous avez besoin se compose de plusieurs commandes, et il sera attendu plusieurs fois tout au long de l'exécution du script. Les fonctions sont définies à l'aide du mot-clé "function" et suivies du corps de la fonction entouré d'accolades.

L'exemple suivant définit une fonction shell simple à utiliser pour imprimer les détails de l'utilisateur et effectue deux appels de fonction, imprimant ainsi les détails de l'utilisateur deux fois lors d'une exécution de script.

```
1 #!/bin/bash
2
3 function user_details {
4     echo "Nom d'utilisateur: $(whoami)"
5     echo "Dossier home: $HOME"
6 }
7
8 user_details
9 user_details
```

BASH - FUNCTIONS

Le nom de la fonction est `user_details`, et le corps de la fonction entre accolades se compose du groupe de deux commandes `echo`. Chaque fois qu'un appel de fonction est effectué en utilisant le nom de la fonction, les deux commandes d'écho dans notre définition de fonction sont exécutées.

Il est important de souligner que la définition de la fonction doit précéder l'appel de fonction, sinon le script renverra une erreur de fonction non trouvée.

```
[benix@beni-xps Bash]$ ./function.sh  
Nom d'utilisateur: benix  
Dossier home: /home/benix  
Nom d'utilisateur: benix  
Dossier home: /home/benix
```


BASH - FUNCTIONS - ECHO

L'exemple précédent a également introduit une autre technique lors de l'écriture de scripts ou de tout programme d'ailleurs, la technique appelée indentation. Les commandes echo dans la définition de la fonction `user_details` ont été délibérément décalées d'un TAB vers la droite, ce qui rend notre code plus lisible et plus facile à résoudre.

Avec l'indentation, il est beaucoup plus clair de voir que les deux commandes ci-dessous renvoient à la définition de la fonction `user_details`.

Il n'y a pas de convention générale sur la façon d'indenter le script bash, il appartient donc à chaque individu de choisir sa propre manière d'indenter. On peut utiliser TAB ou encore, il est parfaitement acceptable d'utiliser à la place un seul TAB 4 espaces, etc.

BASH - FUNCTIONS - ECHO

Ajoutons une nouvelle fonctionnalité à notre script backup.sh existant.

Nous allons programmer deux nouvelles fonctions pour signaler un certain nombre de répertoires et de fichiers à inclure dans le cadre de la sortie compressée du fichier de sauvegarde.

```
[benix@beni-xps Bash]$ ./backup.sh
Les fichiers incluses sont:1
Les dossiers incluses sont:1
La sauvegarde de /home/benix/Documents/Scannes est terminée. Voici quelques détails:
-rw-r--r-- 1 benix benix 455667 31 oct. 15:11 /tmp/benix-Scannes-2021-10-31.tar.gz
```

```
#!/bin/bash

#Ce script va faire une sauvegarde dans le dossier /tmp

user=$(whoami)
input=/home/$user/Documents/Scannes
output=/tmp/${user}-Scannes-$(date +%Y-%m-%d).tar.gz

#la fonction total_files donne un nombre total de fichier dans un dossier.

function total_files {
    find $1 -type f | wc -l
}

#la fonction total_directories montre le nombre total des dossiers.

function total_directories {
    find $1 -type d | wc -l
}

tar -czf $output $input 2> /dev/null

echo -n "Les fichiers incluses sont:"
total_files $input
echo -n "Les dossiers incluses sont:"
total_directories $input

echo "La sauvegarde de $input est terminée. Voici quelques détails:"
ls -l $output

exit $?
```

BASH - FUNCTIONS - ECHO

Après avoir examiné le script backup.sh ci-dessus, vous remarquerez les changements suivants dans le code :

- Nous avons défini une nouvelle fonction appelée `total_files`. La fonction a utilisé les commandes *find* et *wc* pour déterminer le nombre de fichiers situés dans un répertoire qui lui a été fourni lors de l'appel de fonction.
- Nous avons défini une nouvelle fonction appelée `total_directories`. Identique à la fonction `total_files`, elle utilise les mêmes commandes mais elle signale un certain nombre de répertoires dans un répertoire qui lui est fourni lors de l'appel de la fonction.

`wc` = word count

BASH - VALEURS NUMÉRIQUES ET LES CHAÎNES DE CARACTÈRES

Nous allons apprendre quelques bases des comparaisons de shell bash numériques et de chaînes de caractères. En utilisant des comparaisons, nous pouvons comparer des chaînes (mots, phrases) ou des nombres entiers bruts ou sous forme de variables. Voici les opérateurs de comparaison rudimentaires pour les nombres et les chaînes de caractères :

DESCRIPTION	COMPARAISON NUMÉRIQUE	COMPARAISON DE CHAÎNES DE CARACTÈRES
less than (moins que)	-lt	<
greater than (plus grand que)	-gt	>
equal (égale à)	-eq	=
not equal (n'est pas égale à)	-ne	!=
less or equal (inférieur ou égal)	-le	N/A
greater or equal (greater or equal)	-ge	N/A
Exemple de comparaison shell:	[100 -eq 50]; echo \$?	["GNU" = "UNIX"]; echo \$?

BASH - VALEURS NUMÉRIQUES ET LES CHAÎNES DE CARACTÈRES

Nous allons à présent comparer des valeurs numériques telles que deux entiers 1 et 2. L'exemple suivant définira d'abord deux variables \$a et \$b pour contenir nos valeurs entières. Ensuite, nous utilisons des crochets et des opérateurs de comparaison numériques pour effectuer l'évaluation proprement dite. Pour afficher le résultat :

echo \$?

```
[benix@beni-xps Bash]$ a=1
[benix@beni-xps Bash]$ b=2
[benix@beni-xps Bash]$ [ $a -lt $b ]
[benix@beni-xps Bash]$ echo $?
0
```

BASH - VALEURS NUMÉRIQUES ET LES CHAÎNES DE CARACTÈRES

Un ou deux valeurs sont possibles (Vrai ou Faux).

Si la valeur de retour est égale à 0, alors l'évaluation de comparaison est vraie. Cependant, si la valeur de retour est égale à 1, l'évaluation est fausse.

```
[benix@beni-xps Bash]$ a=1
[benix@beni-xps Bash]$ b=2
[benix@beni-xps Bash]$ [ $a -lt $b ]
[benix@beni-xps Bash]$ echo $?
0
[benix@beni-xps Bash]$ [ $a -gt $b ]
[benix@beni-xps Bash]$ echo $?
1
[benix@beni-xps Bash]$ [ $a -eq $b ]
[benix@beni-xps Bash]$ echo $?
1
[benix@beni-xps Bash]$ [ $a -ne $b ]
[benix@beni-xps Bash]$ echo $?
0
```

BASH - VALEURS NUMÉRIQUES ET LES CHAÎNES DE CARACTÈRES

En utilisant des opérateurs de comparaison de chaînes de caractères, nous pouvons également comparer des chaînes de la même manière que lors de la comparaison de valeurs numériques. Ici un autre exemple :

```
[benix@beni-xps Bash]$ [ "voiture" = "camions" ]  
[benix@beni-xps Bash]$ echo $?  
1  
[benix@beni-xps Bash]$ str1="voitures"  
[benix@beni-xps Bash]$ str2="camions"  
[benix@beni-xps Bash]$ [ $str1 = $str2 ]  
[benix@beni-xps Bash]$ echo $?  
1
```

En utilisant l'opérateur de comparaison de chaînes = nous comparons deux chaînes distinctes pour voir si elles sont égales.

BASH - VALEURS NUMÉRIQUES ET LES CHAÎNES DE CARACTÈRES

Si nous devions traduire ceux dont nous avons vu avant en un simple script shell bash, le script ressemblerait à celui illustré ci-dessous. De même, nous comparons deux nombres entiers à l'aide de l'opérateur de comparaison numérique pour déterminer s'ils sont de valeur égale. N'oubliez pas que 0 signale vrai, tandis que 1 indique faux :

```
[benix@beni-xps Bash]$ ./valeurs.sh
Est-ce qu'une voiture et un camion ont le même poids?
0
Est-ce que le 50 est égale à 100 ?
0
Est-ce que rouge and vert sont toutes les deux des couleurs?
1
```

```
#!/bin/bash

string_a="voiture"
string_b="camion"

echo "Est-ce qu'une $string_a et un $string_b ont le même poids? "
[ $string_a != $string_b ]
echo $?

num_a=50
num_b=100

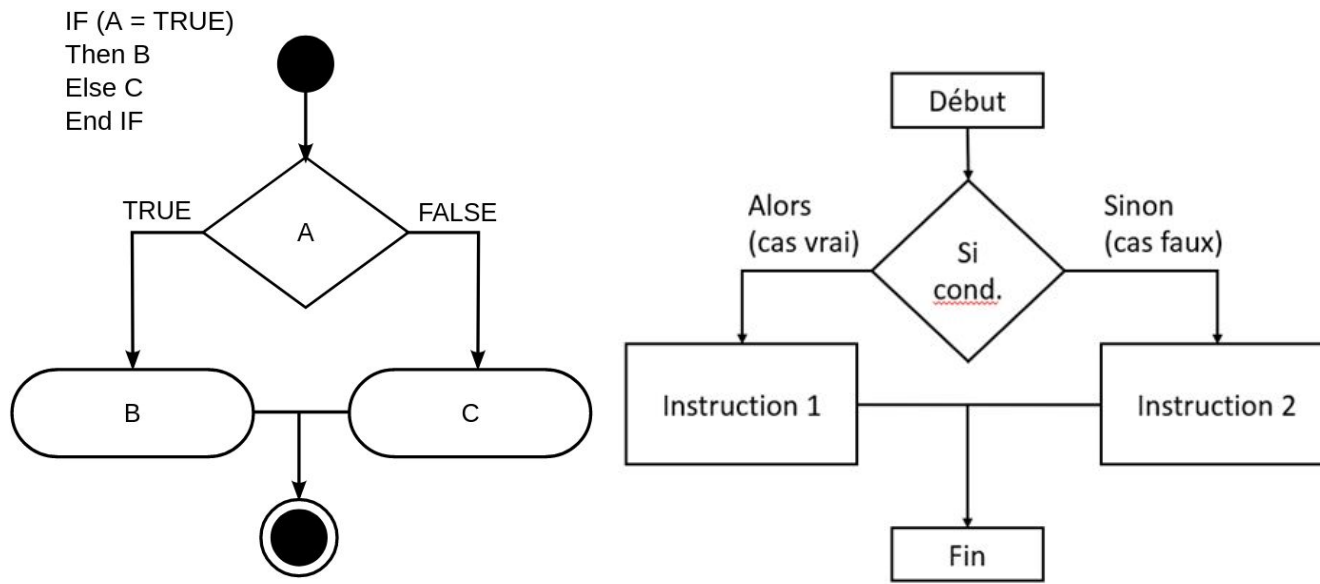
echo "Est-ce que le $num_a est égale à $num_b ?"
[ $num_a -lt $num_b ]
echo $?

color_1="rouge"
color_2="vert"

echo "Est-ce que $color_1 and $color_2 sont toutes les deux des couleurs?"
[ $color_1 = color_2 ]
echo $?
```


BASH - VALEURS NUMÉRIQUES ET LES CHAÎNES DE CARACTÈRES

Les opérations de comparaison auront plus de sens une fois que nous connaîtrons les instructions conditionnelles comme if/else.



BASH - EXPRESSIONS CONDITIONNELLES

Maintenant, il est temps de donner à notre script de sauvegarde une certaine logique en incluant quelques instructions conditionnelles. Les conditions permettent au programmeurs d'implémenter la prise de décision dans un script shell en fonction de certaines conditions ou événements. Les conditionnels auxquels nous faisons référence sont bien sûr, if, then et else.

Par exemple, nous pouvons améliorer notre script de sauvegarde en mettant en œuvre un contrôle de cohérence pour comparer le nombre de fichiers et de répertoires dans un répertoire source que nous avons l'intention de sauvegarder et le fichier de sauvegarde. Le pseudo-code pour ce type d'implémentation se lira comme suit :

SI le nombre de fichiers entre la source et la destination cible est égal ALORS imprimer le message OK, AUTREMENT, imprimer ERREUR.

BASH - EXPRESSIONS CONDITIONNELLES

Commençons par créer un script bash simple décrivant une construction de base if/then/else.

```
#!/bin/bash

num_a=200
num_b=300

if [ $num_a -lt $num_b ]; then
    echo "$num_a est inférieur à $num_b!"
fi
```

Pour l'instant, le conditionnel else a été délibérément omis, nous l'inclurons une fois que nous aurons compris la logique derrière le script ci-dessus.

```
[benix@beni-xps Bash]$ ./Conditional-Statements.sh
200 est inférieur à 300!
```

BASH - EXPRESSIONS CONDITIONNELLES

Les lignes 3 à 4 sont utilisées pour initialiser une variable entière. Sur la ligne 6, nous commençons un bloc conditionnel if ce qui compare les deux variables et si l'évaluation de la comparaison donne vrai, alors à la ligne 7, la commande `echo` nous informera que la valeur dans la variable `$num_a` est inférieure à celle de la variable `$num_b`.

La ligne 8 ferme notre bloc conditionnel if avec un mot-clé fi.

```
1 #!/bin/bash
2
3 num_a=200
4 num_b=300
5
6 if [ $num_a -lt $num_b ]; then
7     echo "$num_a est inférieur à $num_b!"
8 fi
```

BASH - EXPRESSIONS CONDITIONNELLES

Dans le cas où la variable \$num_a est supérieure à \$num_b, notre script ne réagit pas. C'est là que else le conditionnel est utile. Mettez à jour votre script en ajoutant else :

```
1 #!/bin/bash
2
3 num_a=400
4 num_b=300
5
6 if [ $num_a -lt $num_b ]; then
7     echo "$num_a est inférieur à $num_b!"
8 else
9     echo "$num_a est plus grand que $num_b!"
10 fi
```

La ligne 8 contient maintenant la partie else de notre bloc conditionnel. Si l'évaluation de comparaison sur la ligne 6 retourne faux alors le code sous l'instruction else, la ligne 9 est exécutée.

```
[benix@beni-xps Bash]$ ./Conditional-Statements.sh
400 est plus grand que 300!
```

BASH - EXPRESSIONS CONDITIONNELLES

Nous pouvons maintenant améliorer notre script pour effectuer un contrôle de cohérence en comparant la différence entre le nombre total de fichiers avant et après la commande de sauvegarde. Voici le nouveau script backup.sh mis à jour :

```
[benix@beni-xps Bash]$ ./backup.sh
Les fichiers inculs sont: 1
Les répertoires inclus sont: 1
Les fichiers archivés sont; 1
Les répertoires archivés sont: 1
La sauvegarde /home/benix/Documents/Scannes a été terminée!
Voici les détails sur la sauvegarde:
-rw-r--r-- 1 benix benix 455667 31 oct. 17:49 /tmp/benix-Scannes-2021-10-31.tar.gz
[benix@beni-xps Bash]$ ./backup.sh
Les fichiers inculs sont: 2
Les répertoires inclus sont: 2
Les fichiers archivés sont; 2
Les répertoires archivés sont: 2
La sauvegarde /home/benix/Documents/Scannes a été terminée!
Voici les détails sur la sauvegarde:
-rw-r--r-- 1 benix benix 455815 31 oct. 17:50 /tmp/benix-Scannes-2021-10-31.tar.gz
```

```
1 #!/bin/bash
2
3 #Ce script va faire une sauvegarde dans le dossier /tmp
4
5 user=$(whoami)
6 input=/home/$user/Documents/Scannes
7 output=/tmp/${user}-Scannes-$(date +%Y-%m-%d).tar.gz
8
9 #la fonction total_files donne un nombre total de fichier dans un dossier.
10
11 function total_files {
12     find $1 -type f | wc -l
13 }
14
15 #la fonction total_directories montre le nombre total des dossiers.
16
17 function total_directories {
18     find $1 -type d | wc -l
19 }
20
21 #la fonction total_archived_directories montre le nombre total de répertoires.
22
23 function total_archived_directories {
24     tar -tzf $1 | grep /$ | wc -l
25 }
26
27 #la fonction total_archived_files montre le nombre total de fichiers.
28
29 function total_archived_files {
30     tar -tzf $1 | grep -v /$ | wc -l
31 }
32
33 tar -czf $output $input 2> /dev/null
34
35 src_files=$( total_files $input )
36 src_directories=$( total_directories $input )
37
38 arch_files=$( total_archived_files $output )
39 arch_directories=$( total_archived_directories $output )
40
41 echo "Les fichiers inculs sont: $src_files"
42 echo "Les répertoires inclus sont: $src_directories"
43 echo "Les fichiers archivés sont; $arch_files"
44 echo "Les répertoires archivés sont: $arch_directories"
45
46 if [ $src_files -eq $arch_files ]; then
47     echo "La sauvegarde $input a été terminée!"
48     echo "Voici les détails sur la sauvegarde:"
49     ls -l $output
50 else
51     echo "La sauvegarde de $output a été échouée!"
52 fi
```

BASH - PARAMÈTRES DE POSITION

Nous pouvons compter le nombre de fichiers et de répertoires inclus dans le fichier de sauvegarde compressé.

De plus, notre script facilite également une vérification de l'intégrité pour confirmer que tous les fichiers ont été correctement sauvegardés. L'inconvénient est qu'on est toujours obligé de sauvegarder un répertoire d'un utilisateur courant.

Ce serait formidable si le script était suffisamment flexible pour permettre à l'administrateur système de sauvegarder un répertoire personnel de tout utilisateur système sélectionné en pointant simplement le script vers son répertoire personnel.

BASH - PARAMÈTRES DE POSITION

Lorsque vous utilisez des paramètres de position en bash, c'est plutôt une tâche facile. Les paramètres de position sont affectés via des arguments de ligne de commande et sont accessibles dans un script en comme variables \$1, \$2...\$N. Pendant l'exécution du script, tous les éléments supplémentaires fournis après le nom du programme sont considérés comme des arguments et sont disponibles pendant l'exécution du script.

Sur la ligne 3, nous affichons les paramètres de position 1er, 2e et 4e exactement dans l'ordre où ils sont fournis lors de l'exécution du script. Le 3ème paramètre est disponible, mais volontairement omis sur cette ligne. En utilisant \$# sur la ligne 4, nous affichons le nombre total d'arguments fournis. Ceci est utile lorsque nous devons vérifier le nombre d'arguments fournis par l'utilisateur lors de l'exécution du script.

```
1 #!/bin/bash
2
3 echo $1 $2 $4
4 echo $#
5 echo $*
```


BASH - PARAMÈTRES DE POSITION

Enfin, le `$*` sur la ligne 5, est utilisé pour imprimer tous les arguments.

```
[benix@beni-xps Bash]$ ./param.sh 1 2 3 4
1 2 4
4
1 2 3 4
[benix@beni-xps Bash]$ ./param.sh hello you !
hello you
3
hello you !
```

```
1 #!/bin/bash
2
3 echo $1 $2 $4
4 echo $#
5 echo $*
```

Améliorons maintenant notre script `backup.sh` pour accepter les arguments d'une ligne de commande. Ce que nous recherchons ici, c'est de laisser l'utilisateur décider quel répertoire sera sauvegardé. Dans le cas où aucun argument n'est soumis par l'utilisateur pendant l'exécution du script, par défaut, le script sauvegardera le répertoire personnel d'un utilisateur actuel.

BASH - PARAMÈTRES DE POSITION

La mise à jour du script backup.sh introduit quelques nouvelles techniques, mais le reste du code entre les lignes 5 et 13 devrait maintenant être explicite. La ligne 5 utilise une option bash -z en combinaison avec une instruction conditionnelle if pour vérifier si le paramètre positionnel \$1 contient une valeur. -z renvoie simplement *vrai* si la longueur de la chaîne qui dans notre cas est la variable \$1 est nulle. Si tel est le cas, nous définissons la variable \$user sur le nom d'un utilisateur actuel.

```
1 #!/bin/bash
2
3 #Ce script va faire une sauvegarde dans le dossier /tmp
4
5 if [ -z $1 ]; then
6     user=$(whoami)
7 else
8     if [ ! -d "/home/$1" ]; then
9         echo "Le répertoire de base de l'utilisateur $1 demandé n'existe pas."
10        exit 1
11    fi
12    user=$1
13 fi
14
15 #user=$(whoami)
16 input=/home/$user/Documents/Scannes
17 output=/tmp/${user}-Scannes-$(date +%Y-%m-%d).tar.gz
18
19 #la fonction total files donne un nombre total de fichier dans un dossier.
```

```
[benix@beni-xps Bash]$ ./backup.sh
Les fichiers inculs sont: 2
Les répertoires inclus sont: 2
Les fichiers archivés sont: 2
Les répertoires archivés sont: 2
La sauvegarde /home/benix/Documents/Scannes a été terminée!
Voici les détails sur la sauvegarde:
-rw-r--r-- 1 benix benix 455815 1 nov. 00:54 /tmp/benix-Scannes-2021-11-01.tar.gz
[benix@beni-xps Bash]$ ./backup.sh yellow
Le répertoire de base de l'utilisateur yellow demandé n'existe pas.
[benix@beni-xps Bash]$ ./backup.sh benix
Les fichiers inculs sont: 2
Les répertoires inclus sont: 2
Les fichiers archivés sont: 2
Les répertoires archivés sont: 2
La sauvegarde /home/benix/Documents/Scannes a été terminée!
Voici les détails sur la sauvegarde:
-rw-r--r-- 1 benix benix 455815 1 nov. 00:55 /tmp/benix-Scannes-2021-11-01.tar.gz
```

BASH - PARAMÈTRES DE POSITION

Sinon, à la ligne 8, nous vérifions si le répertoire personnel de l'utilisateur demandé existe en utilisant l'option `-d`. Notez le point d'exclamation avant l'option `-d`. Le point d'exclamation, dans ce cas, agit comme un négateur. Par défaut, l'option `-d` renvoie true si le répertoire existe, d'où notre `!` inverse simplement la logique et sur la ligne 9, nous affichons un message d'erreur.

La ligne 10 utilise la commande `exit` provoquant l'arrêt de l'exécution du script. Nous avons également attribué la valeur de sortie 1 au lieu de 0, ce qui signifie que le script s'est terminé avec une erreur. Si la vérification du répertoire réussit la validation, à la ligne 12, nous attribuons notre variable `$user` au paramètre de position `$1` à la demande de l'utilisateur.

BASH - BOUCLES (LOOPS)

Jusqu'à présent, notre script de sauvegarde fonctionne comme prévu. Nous pouvons désormais facilement sauvegarder n'importe quel répertoire utilisateur en pointant le script vers le répertoire personnel de l'utilisateur à l'aide de "paramètres de position" lors de l'exécution du script.

Le problème ne survient que lorsque nous devons sauvegarder quotidiennement plusieurs répertoires d'utilisateurs. Cette tâche deviendra donc très vite fastidieuse et chronophage. il serait plus intéressant d'avoir les moyens de sauvegarder n'importe quel nombre de répertoires personnels d'utilisateurs sélectionnés avec une seule exécution de script backup.sh.

```
#!/bin/bash
# For loop with individual numbers
for i in 0 1 2 3 4 5
do
    echo "Element $i"
done
```

BASH - BOUCLES (LOOPS)

Cette tâche peut être accomplie en utilisant des boucles. Les boucles sont des “constructions en boucle” utilisées pour parcourir un nombre donné de tâches jusqu'à ce que tous les éléments d'une liste spécifiée soient terminés ou que des conditions prédéfinies soient remplies. Trois types de boucles de base sont à notre disposition :

- For
- While
- Until

```
until loop  
10 is not equal to 1  
9 is not equal to 1  
8 is not equal to 1  
7 is not equal to 1  
6 is not equal to 1  
5 is not equal to 1  
4 is not equal to 1  
3 is not equal to 1  
2 is not equal to 1  
i value is 1  
loop terminated
```

BASH - BOUCLES (LOOPS) - FOR

La boucle For est utilisée pour parcourir n'importe quel code donné pour n'importe quel nombre d'éléments fournis dans la liste. Commençons par un exemple simple de boucle for :

```
[benix@beni-xps Bash]$ for i in 1 2 3; do echo $i; done  
1  
2  
3
```

La boucle for ci-dessus a utilisé la commande echo pour imprimer tous les éléments 1, 2 et 3 de la liste. L'utilisation d'un point-virgule nous permet d'exécuter une boucle for sur une seule ligne de commande.

BASH - BOUCLES (LOOPS) - FOR

Si nous devons transférer la boucle for ci-dessus dans un script bash, le code ressemblerait à ceci :

```
#!/bin/bash

for i in 1 2 3; do
    echo $i
done
```

La boucle for se compose de quatre mots Shell réservés : for, in, do, done. Le code ci-dessus peut donc également être lu comme : POUR chaque élément DANS les listes 1, 2 et 3, affectez temporairement chaque élément dans une variable i, après quoi FAIRE un echo \$i afin d'imprimer l'élément en tant que STDOUT et de continuer à imprimer jusqu'à ce que tous les éléments de la liste soient TERMINÉS.

BASH - BOUCLES (LOOPS) - FOR

Imprimer des chiffres est sans aucun doute amusant, mais essayons plutôt quelque chose de plus significatif. En utilisant la substitution de commande comme expliqué précédemment, nous pouvons créer n'importe quel type de liste pour faire partie de la construction de la boucle for. L'exemple de boucle for suivant légèrement plus sophistiqué comptera les caractères de chaque ligne pour un fichier donné :

```
[benix@beni-xps Bash]$ vi items.txt
[benix@beni-xps Bash]$ cat items.txt
bash
scripting
tutorial
[benix@beni-xps Bash]$ for i in $( cat items.txt ); do echo -n $i | wc -c; done
4
9
8
```


BASH - BOUCLES (LOOPS) - WHILE

La construction de boucle suivante sur notre liste est while loop. Cette boucle particulière agit sur une condition donnée. Cela signifie qu'il continuera à exécuter le code entre DO et DONE tant que la condition spécifiée est vraie. Une fois que la condition spécifiée devient fausse, l'exécution s'arrête. Exemple :

Cette boucle while particulière continuera à exécuter le code inclus uniquement tant que la variable "counter" est inférieure à 3. Cette condition est définie sur la ligne 4. Au cours de chaque itération de la boucle, sur les lignes 5, la variable "counter" est incrémentée de un. Dès que la variable "counter" est égale à 3, la condition définie sur les lignes 4 devient fausse, tandis que l'exécution de la boucle est terminée.

```
1  #!/bin/bash
2
3  counter=0
4  while [ $counter -lt 3 ]; do
5      let counter+=1
6      echo $counter
7  done
```

BASH - BOUCLES (LOOPS) - WHILE

```
[benix@beni-xps Bash]$ cat while.sh
#!/bin/bash

counter=0
while [ $counter -lt 3 ]; do
    let counter+=1
    echo $counter
done
[benix@beni-xps Bash]$ ./while.sh
1
2
3
```



```
#!/bin/bash

counter=2
while [ $counter -lt 3 ]; do
    let counter+=1
    echo $counter
done
```



```
[benix@beni-xps Bash]$ ./while.sh
3
```

BASH - BOUCLES (LOOPS) - UNTIL

La dernière boucle que nous allons voir est UNTIL. La boucle UNTIL fait exactement le contraire de la boucle WHILE. Jusqu'à ce que la boucle agisse également sur une condition prédéfinie. Cependant, le code compris entre DO et DONE est exécuté de manière répétée uniquement jusqu'à ce que cette condition passe de faux à vrai. Exemple:

Si vous avez compris le script de boucle while ci-dessus, la boucle UNTIL sera quelque peu explicite. Le script démarre avec la variable "counter" définie sur la ligne 6. La condition définie à la ligne 4 de cette boucle particulière UNTIL est de continuer à exécuter le code inclus jusqu'à ce que la condition devienne vraie.

```
1  #!/bin/bash
2
3  counter=6
4  until [ $counter -lt 3 ]; do
5      let counter-=1
6      echo $counter
7  done
```

BASH - BOUCLES (LOOPS) - UNTIL

```
[benix@beni-xps Bash]$ cat until.sh
#!/bin/bash

counter=6
until [ $counter -lt 3 ]; do
    let counter-=1
    echo $counter
done
[benix@beni-xps Bash]$ ./until.sh
5
4
3
2
```



```
#!/bin/bash

counter=4
until [ $counter -lt 3 ]; do
    let counter-=1
    echo $counter
done
```



```
[benix@beni-xps Bash]$ ./until.sh
3
2
```

BASH

À ce stade, nous pouvons convertir notre compréhension des boucles en quelque chose de tangible. Notre script de sauvegarde actuel est actuellement capable de sauvegarder un seul répertoire par exécution. Ce serait bien d'avoir la possibilité de sauvegarder tous les répertoires fournis au script sur une ligne de commande lors de son exécution.

Vous pouvez toutefois vous renseigner sur le fonctionnement Bash Arithmétique.

L'arithmétique dans les scripts bash ajoutera un autre niveau de sophistication et de flexibilité à nos scripts car elle nous permet de calculer des nombres même avec une précision numérique. Il existe plusieurs façons d'accomplir des opérations arithmétiques dans vos scripts bash (les commandes suivants : `expr` , `let` , `bc`

A vous de jouer !

BASH - EXEMPLE SCRIPT SAUVEGARDE

```

1  #!/bin/bash
2
3  # This bash script is used to backup a user's home directory to /tmp/.
4
5  function backup {
6
7      if [ -z $1 ]; then
8          user=$(whoami)
9      else
10         if [ ! -d "/home/$1" ]; then
11             echo "Requested $1 user home directory doesn't exist."
12             exit 1
13         fi
14         user=$1
15     fi
16
17     input=/home/$user
18     output=/tmp/${user}_home_$(date +%Y-%m-%d_%H%M%S).tar.gz
19
20     function total_files {
21         find $1 -type f | wc -l
22     }
23
24     function total_directories {
25         find $1 -type d | wc -l
26     }
27
28     function total_archived_directories {
29         tar -tzf $1 | grep /$ | wc -l
30     }
31
32     function total_archived_files {
33         tar -tzf $1 | grep -v /$ | wc -l
34     }
35
36     tar -czf $output $input 2> /dev/null
37
38     src_files=$( total_files $input )
39     src_directories=$( total_directories $input )
40
41     arch_files=$( total_archived_files $output )
42     arch_directories=$( total_archived_directories $output )
43
44     echo "##### $user #####"
45     echo "Files to be included: $src_files"
46     echo "Directories to be included: $src_directories"
47     echo "Files archived: $arch_files"
48     echo "Directories archived: $arch_directories"
49
50     if [ $src_files -eq $arch_files ]; then
51         echo "Backup of $input completed!"
52         echo "Details about the output backup file:"
53         ls -l $output
54     else
55         echo "Backup of $input failed!"
56     fi
57 }
58
59 for directory in $*; do
60     backup $directory
61 done;

```

BASH

Il n'y a pas l'ensemble des éléments de scripting en bash dans ce cours. Il est important de développer ses compétences dans le scripting en bash et apprendre davantage. La recherche sur Google, il existe d'autres ressources disponibles en ligne pour vous aider si vous êtes bloqué. Le plus important et le plus recommandé de tous est le manuel de référence [Bash de GNU](https://www.gnu.org/software/bash/)

www.gnu.org/software/bash/.

FIN